

DevSecOps CI/CD Pipeline Implementation for Task Manager API

Project Report

Student Name: Mayank Gupta

Student ID: 23BCS10069

Repository: https://github.com/mayank1365/task_manager_api

1. Problem Background & Motivation

1.1 Challenges in Modern Software Delivery

Modern software development faces critical challenges when CI/CD automation is absent:

- **Manual Testing Overhead:** Developers spend significant time running repetitive tests locally, delaying feedback cycles.
- **Late-Stage Security Discovery:** Security vulnerabilities discovered in production are exponentially more expensive to fix than those caught during development.
- **Inconsistent Build Environments:** “Works on my machine” syndrome leads to deployment failures due to environment drift.
- **Deployment Risk:** Manual deployments are error-prone and lack audit trails, making rollbacks difficult.

1.2 The Need for DevSecOps

Traditional CI/CD pipelines treat security as a final gate before production. This approach has fundamental flaws:

- Security teams become bottlenecks, slowing release velocity
- Vulnerabilities discovered late require expensive code rewrites
- Developers lack immediate feedback on security issues

DevSecOps addresses this by implementing **shift-left security**—integrating security checks directly into the automated pipeline. Every code commit is

automatically scanned for vulnerabilities, ensuring security is enforced continuously rather than as an afterthought.

1.3 Project Objective

This project demonstrates a production-grade DevSecOps CI/CD pipeline that:

1. Automates the entire build-test-deploy lifecycle
 2. Enforces security gates at multiple layers (code, dependencies, container)
 3. Provides fast feedback to developers (fail-fast principle)
 4. Ensures only verified, secure artifacts reach the registry
-

2. Application Overview

2.1 Purpose and Scope

The **Task Manager API** is a RESTful web service for managing tasks (create, list, delete operations). The application is intentionally kept simple to focus attention on the CI/CD pipeline rather than complex business logic.

2.2 Technology Stack

Component	Technology	Version
Programming Language	Java	17
Framework	Spring Boot	3.5.9
Build Tool	Maven	3.x
Container Runtime	Docker	Latest
CI/CD Platform	GitHub Actions	N/A

2.3 Why Simplicity Matters

The application's simplicity serves a pedagogical purpose:

- **Focus on Pipeline:** Complex application logic would obscure pipeline design decisions
 - **Fast Execution:** Simple codebase allows rapid pipeline iterations during development
 - **Clear Failure Signals:** When a pipeline stage fails, it's immediately obvious whether the issue is in the application or the pipeline configuration
-

3. CI/CD Architecture Overview

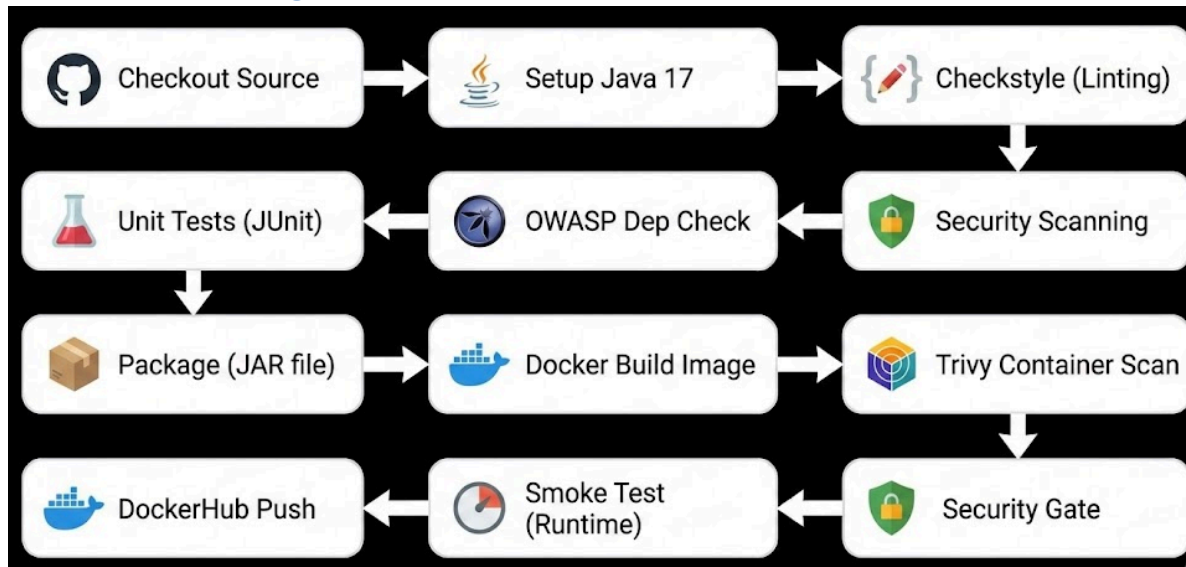
3.1 Workflow Trigger Mechanism

The pipeline is triggered by:

- **Push events** to the main branch (automatic)
- **Manual dispatch** via GitHub Actions UI (on-demand)

This dual-trigger approach balances automation with developer control.

3.2 Architecture Diagram



3.3 Pipeline Philosophy

The pipeline follows three core principles:

1. **Fail Fast:** Cheap checks (linting) run first; expensive checks (container scanning) run later
2. **Security Gates:** No artifact proceeds to the next stage if security checks fail
3. **Immutable Artifacts:** The Docker image built in the pipeline is the exact artifact deployed to production

4. CI/CD Pipeline Design & Stage-Wise Explanation

4.1 Stage 1: Checkout Source

What it does:

- **name:** Checkout Source
- **uses:** actions/checkout@v4

Clones the repository code into the GitHub Actions runner environment.

Why it exists:

All subsequent stages require access to source code, build configurations (pom.xml), and the Dockerfile.

Risk mitigated:

Ensures the pipeline operates on the exact commit that triggered it, preventing race conditions.

4.2 Stage 2: Setup Java 17

What it does:

```
- name: Setup Java 17
  uses: actions/setup-java@v4
  with:
    distribution: 'temurin'
    java-version: '17'
    cache: maven
```

Installs Java 17 (Eclipse Temurin distribution) and configures Maven dependency caching.

Why it exists:

Spring Boot 3.x requires Java 17. Caching Maven dependencies reduces build time by ~30-40% on subsequent runs.

Risk mitigated:

Eliminates “works on my machine” issues by standardizing the Java runtime version.

4.3 Stage 3: Run Checkstyle (Linting)

What it does:

```
- name: Run Checkstyle (Linting)
  run: mvn checkstyle:check
```

Validates code against Google Java Style Guide.

Why it exists:

Code quality issues (inconsistent formatting, unused imports) are the cheapest to detect. Running this first provides immediate feedback.

Risk mitigated:

Prevents technical debt accumulation and enforces team coding standards.

4.4 Stage 4: Dependency Vulnerability Scan (SCA)

What it does:

- `name: Dependency Vulnerability Scan (SCA)`
`continue-on-error: true`
`run: mvn org.owasp:dependency-check-maven:check`

Scans Maven dependencies against the National Vulnerability Database (NVD) for known CVEs.

Why it exists:

Modern applications use 80%+ third-party libraries. A single vulnerable dependency (e.g., Log4Shell) can compromise the entire application.

Risk mitigated:

Detects supply chain attacks and known vulnerabilities in dependencies before they reach production.

Note: `continue-on-error: true` is set because the NVD API has strict rate limits. Trivy provides backup coverage for dependency scanning.

4.5 Stage 5: Run Unit Tests

What it does:

- `name: Run Unit Tests`
`run: mvn test`

Executes JUnit test suite to verify business logic correctness.

Why it exists:

Automated tests catch regressions immediately. If tests fail, there's no point building artifacts.

Risk mitigated:

Prevents broken functionality from reaching production.

4.6 Stage 6: Package Application

What it does:

- **name:** Package Application
 run: `mvn clean package -DskipTests`

Compiles source code and packages it into an executable JAR file.

Why it exists:

Creates the deployable artifact. Tests are skipped here because they already passed in Stage 5.

Risk mitigated:

Ensures the build process itself doesn't introduce errors.

4.7 Stage 7: Build Docker Image

What it does:

- **name:** Build Docker Image
 run: `docker build -t ${ secrets.DOCKERHUB_USERNAME }/task-manager-api:latest .`

Packages the JAR file into a Docker container using the Dockerfile.

Why it exists:

Containers provide environment consistency. The same image runs identically in development, staging, and production.

Risk mitigated:

Eliminates deployment failures caused by environment differences.

Dockerfile Security Enhancement:

```
RUN apt-get update && apt-get upgrade -y && rm -rf  
/var/lib/apt/lists/*
```

This line patches OS-level vulnerabilities in the base image, addressing CVEs in system packages like gnupg.

4.8 Stage 8: Scan Docker Image with Trivy

What it does:

```
- name: Scan Docker Image with Trivy
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: '${{ secrets.DOCKERHUB_USERNAME
}}/task-manager-api:latest'
    severity: 'HIGH,CRITICAL'
    exit-code: '1'
```

Scans the Docker image for vulnerabilities in: - OS packages (Debian/Ubuntu base image) - Application dependencies (JAR files) - Embedded libraries

Why it exists:

Container images bundle the entire runtime environment. A vulnerability in the base OS layer can be exploited even if application code is secure.

Risk mitigated:

Prevents deployment of images with known exploits. `exit-code: '1'` ensures the pipeline fails if HIGH/CRITICAL vulnerabilities are found.

Real-World Example:

During development, Trivy detected CVE-2024-38819 (Path Traversal vulnerability in Spring WebMVC 6.1.x). This led to upgrading Spring Boot from 3.2.1 to 3.5.9, demonstrating the pipeline's effectiveness.

4.9 Stage 9: Container Runtime Smoke Test

What it does:

```
- name: Container Runtime Smoke Test
  run: |
    docker run -d -p 8080:8080 --name test-app ${{
secrets.DOCKERHUB_USERNAME }}/task-manager-api:latest
    sleep 10
    curl -f http://localhost:8080/health
```

Starts the container and verifies the application responds to HTTP requests.

Why it exists:

A container can build successfully but fail at runtime (e.g., missing environment variables, port conflicts).

Risk mitigated:

Catches runtime errors before the image is published to the registry.

4.10 Stage 10: DockerHub Login & Push

What it does:

- **name:** DockerHub Login
run: `echo "${ secrets.DOCKERHUB_TOKEN }}" | docker login -u ${ secrets.DOCKERHUB_USERNAME } --password-stdin`
- **name:** Push Image to DockerHub
run: `docker push ${ secrets.DOCKERHUB_USERNAME }/task-manager-api:latest`

Authenticates with DockerHub and publishes the verified image.

Why it exists:

Makes the image available for deployment to production environments.

Risk mitigated:

Only images that passed all security gates reach the registry. Credentials are stored as GitHub Secrets, never hardcoded.

5. Security & Quality Controls (DevSecOps)

5.1 Shift-Left Security Implementation

Traditional pipelines perform security checks after deployment. This project implements **shift-left security** by integrating checks at every stage:

Stage	Security Layer	Tool	What It Detects
Code	Static Analysis	Checkstyle	Code quality issues
Dependencies	SCA	OWASP Dependency Check	Vulnerable libraries
Container	Image Scanning	Trivy	OS + App vulnerabilities
Runtime	Smoke Test	curl	Runtime failures

5.2 SAST vs SCA vs Container Scanning

Software Composition Analysis (SCA):

Scans third-party dependencies declared in `pom.xml`. Example: Detects if `spring-boot-starter-web` has a known CVE.

Container Scanning:

Scans the entire Docker image, including: - Base OS packages (apt packages in Debian) - Application JARs - Embedded libraries

Why Both Are Needed:

SCA catches dependency issues early (before building). Container scanning provides a final verification of the complete artifact, including OS-level vulnerabilities.

5.3 Security Gate Enforcement

The pipeline enforces a critical rule: **No artifact proceeds if security checks fail.**

Example: If Trivy detects a CRITICAL vulnerability, the pipeline exits with code 1, preventing the image from being pushed to DockerHub. This ensures only verified, secure images reach production.

6. Results & Observations

6.1 Pipeline Execution Behavior

Successful Run:

When all stages pass, the pipeline completes in approximately 3-4 minutes: - Checkstyle: ~10 seconds - OWASP Scan: ~60 seconds (or skipped due to NVD rate limits) - Unit Tests: ~15 seconds - Docker Build: ~40 seconds - Trivy Scan: ~30 seconds - Smoke Test: ~15 seconds

6.2 Fail-Fast Principle in Action

The pipeline is designed to fail as early as possible:

1. If Checkstyle fails (e.g., code formatting issue), the pipeline stops immediately—no resources wasted on building/testing.
2. If unit tests fail, Docker build is skipped.
3. If Trivy detects vulnerabilities, the image is never pushed.

This saves CI/CD costs and provides rapid developer feedback.

6.3 OWASP Dependency Check Behavior

The OWASP stage often shows “BUILD FAILURE” in logs due to NVD API rate limiting (403/404 errors). However, `continue-on-error: true` allows the pipeline to proceed. This is a pragmatic decision:

- **Rationale:** GitHub Actions runners share IP addresses, frequently hitting NVD rate limits

- **Mitigation:** Trivy provides comprehensive dependency scanning as backup
 - **Future Fix:** Obtain an NVD API key and configure it as `secrets.NVD_API_KEY`
-

7. Limitations & Future Improvements

7.1 Current Limitations

1. **Single Environment:** The pipeline only pushes to DockerHub. No staging/production separation.
2. **No Continuous Deployment:** Requires manual deployment from DockerHub to production.
3. **Limited Observability:** No integration with monitoring tools (Prometheus, Grafana).
4. **Basic Smoke Test:** Only verifies `/health` endpoint; doesn't test full API functionality.

7.2 Proposed Improvements

1. Multi-Environment Deployment

Extend the pipeline to deploy to staging automatically and production on manual approval:

- `name: Deploy to Staging`
`if: github.ref == 'refs/heads/main'`
`run: kubectl apply -f k8s/staging/`
- `name: Deploy to Production`
`if: github.event_name == 'workflow_dispatch'`
`run: kubectl apply -f k8s/production/`

2. Image Signing

Use Cosign to cryptographically sign Docker images, ensuring supply chain integrity:

- `name: Sign Image`
`run: cosign sign ${ secrets.DOCKERHUB_USERNAME }/task-manager-api:latest`

3. Policy-as-Code

Integrate Open Policy Agent (OPA) to enforce organizational policies (e.g., “no images with CRITICAL CVEs”):

- `name: Policy Check`
`run: conftest test Dockerfile --policy policy/`

4. Comprehensive Integration Tests

Replace basic smoke test with full API test suite using Postman/Newman:

```
- name: API Integration Tests  
  run: newman run tests/api-collection.json
```

8. Conclusion

This project demonstrates a production-grade DevSecOps CI/CD pipeline that addresses real-world software delivery challenges. Key achievements include:

1. **Automated Security:** Vulnerabilities are detected automatically at multiple layers (code, dependencies, container), reducing security debt.
2. **Fast Feedback:** Developers receive immediate feedback on code quality and security issues, enabling rapid iteration.
3. **Immutable Artifacts:** Docker images ensure environment consistency from development to production.
4. **Fail-Fast Design:** The pipeline optimizes resource usage by running cheap checks first and stopping immediately on failure.

Key Learnings

- **Security is not a checkbox:** Integrating security into every pipeline stage (shift-left) is more effective than final-stage audits.
- **Tooling matters:** Trivy's comprehensive scanning caught a critical Spring Boot vulnerability that would have reached production without automated checks.
- **Pragmatism over perfection:** The continue-on-error configuration for OWASP demonstrates that real-world pipelines must balance ideal practices with operational constraints.

Final Reflection

The DevSecOps pipeline transforms security from a bottleneck into an enabler. By automating security gates, teams can deploy faster *and* more securely—a paradigm shift from the traditional trade-off between speed and safety. This project serves as a blueprint for building secure, automated software delivery pipelines in modern cloud-native environments.
